PremièreImpression

Dossier de lancement — M1 MIAGE Apprentissage — v0.1.0-SNAPSHOT

Alice O'Péhidémerveille, Bob Léponje, Charles McGill, David Copperfield · 20 octobre 2026

01 Équipe, comptes développeurs et dépôt

MEMBRE	COMPTE GITHUB	EMAIL DES COMMITS (`GIT CONFIG USER.EMAIL`)
Alice O'Péhidémerveille	`@alice-exemple`	`alice@exemple.invalid`
Bob Léponje	`@bob-exemple`	`bob@exemple.invalid`
Charles McGill	`@charles-exemple`	`charles@exemple.invalid`
David Copperfield	`@david-exemple`	`david@exemple.invalid`

Équipe fictive : ce dossier est un exemple réalisé par l'enseignant avec une IA, pour illustrer les attendus. Chaque membre a vérifié que son email Git est associé à son compte GitHub, pour que ses commits lui soient bien attribués. **Dépôt unique** pour les quatre couches : https://github.com/pascalpoizat/premiere-impression-exemple · **page de projet** : https://pascalpoizat.github.io/premiere-impression-exemple/.

Organisation : pas de silos. Chaque membre porte une feature complète par itération, de la BD au front. Les tâches transverses (intégration continue, qualité, migrations, front, dossier, livraisons) sont partagées par toute l'équipe, et chaque pull request est relue par un autre membre.

02 Projet et caractère innovant

Le problème. Pour un étudiant ou un apprenti, l'entretien (de stage, d'alternance ou d'embauche) est un moment décisif. Pourtant, sa préparation est éclatée entre de nombreux outils : les annonces sur les sites d'emploi, les rendez-vous dans un agenda, les révisions sur papier, l'itinéraire sur une application de transport. Quant à la tenue, beaucoup d'étudiants n'en possèdent pas une adaptée et n'ont pas les moyens de l'acheter pour un seul entretien.

PremièreImpression rassemble tout ce qui aide à faire bonne impression, autour d'un objet central : **la candidature et son entretien**. Chaque feature répond à la même question : « qu'est-ce qui m'aide à réussir cet entretien-là ? »

Ce qui est innovant :

- **Le calcul plutôt que le stockage.** L'application ne se contente pas de ranger des informations, elle produit des décisions : quelles compétences l'annonce attend et lesquelles me manquent, quels vêtements me vont vraiment, quoi réviser chaque jour jusqu'au jour J, à quelle heure partir. Chaque feature repose sur un algorithme identifié (voir le plan des features).
- **Des features qui s'alimentent entre elles.** L'analyse de l'annonce repère les compétences manquantes, qui deviennent le programme de révision. La date de l'entretien

sert d'échéance à la fois au planning de révision, au prêt de tenue et au calcul de l'heure de départ.

- **Une dimension solidaire.** Les étudiants se prêtent des tenues adaptées aux entretiens, et un **ajustement morphologique** fondé sur les mensurations, pas seulement sur la taille de l'étiquette, évite les prêts inutiles.
- **Ce qui existe déjà.** Les sites d'emploi suivent les candidatures, les plateformes de seconde main vendent des vêtements, les applications de révision gèrent des fiches. Aucun outil ne relie ces besoins autour de l'entretien.

03 Scénarios d'usage

1. En tant que candidat ou candidate, je colle le texte d'une annonce. **L'application en extrait les compétences attendues et calcule ma correspondance avec mon profil**, afin de savoir sur quoi insister et quoi travailler.
2. En tant que candidat ou candidate, je saisis mes mensurations. **L'application me propose les vêtements prêtés par d'autres étudiants qui m'iront réellement**, classés par qualité d'ajustement.
3. En tant que candidat ou candidate, à partir des compétences à travailler et de mes disponibilités, **j'obtiens un planning de révision jusqu'à la date de l'entretien**, qui privilégie ce qui rapporte le plus.
4. En tant que candidat ou candidate, **je connais l'heure à laquelle partir** pour arriver à l'heure, avec une marge de sécurité.
5. En tant que prêteur ou prêteuse, je déclare un vêtement avec ses mesures. **Ses disponibilités sont calculées automatiquement** à partir des prêts déjà réservés et des délais de nettoyage.
6. En tant que candidat ou candidate qui a plusieurs entretiens, **je suis prévenu des conflits d'agenda**, y compris lorsque le temps de trajet entre deux entretiens est insuffisant.

04 Plan des features

Une feature a un effet sur **la BD, le back, l'API REST et le front**. Chaque feature est portée par **une seule personne** et repose sur **un algorithme** identifié.

VERSION	ALICE (CANDIDATURE)	BOB (TENUE)	CHARLES (PRÉPARATION)	DAVID (PONCTUALITÉ)
v0.1.0 24/11	Analyse d'annonce et correspondance avec le profil — TF-IDF, similarité cosinus	Ajustement morphologique — distance pondérée asymétrique, top-k par tas	Planning de révision jusqu'au jour J — sac à dos sous contrainte de temps	Heure de départ — Connection Scan Algorithm
v0.2.0 08/12	Conflits d'agenda avec temps de trajet — ordonnancement d'intervalles	Calendrier des prêts avec délais de nettoyage — arbre d'intervalles	Mémorisation des notions — répétition espacée SM-2	Marge de sécurité tirée de l'historique des retards — quantiles empiriques
v1.0.0 19/01	Parcours d'apprentissage pour combler l'écart de compétences — tri topologique des prérequis	Composition de tenue — contraintes, backtracking	Simulation d'entretien adaptative — classement Elo des questions et du candidat	Répartition des vêtements entre plusieurs entretiens — algorithme hongrois

05 Architecture

COUCHE	CHOIX	REMARQUES
Front	application web : HTML, CSS, JavaScript natif	client de l'API REST uniquement ; dossier `front/`
API REST	Spring Boot 4 (Spring MVC), OpenAPI / Swagger UI	versionnée sous `/api/v1`, erreurs au format RFC 9457
Back	Java 21, Spring Boot 4, Spring Data JPA, Bean Validation	**architecture hexagonale**
BD	PostgreSQL 17 (H2 pour les tests)	schéma versionné par **Flyway**, une migration par feature
Qualité et livraison	JUnit 5, AssertJ, ArchUnit, JaCoCo, Codecov, SonarCloud ; GitHub Actions	CI à chaque push ; à chaque tag, **release automatique** (jar + dossier PDF construit par la CI) ; page de projet sur GitHub Pages
Conteneurs	Docker Compose	BD dès v0.1.0, application en v0.2.0

Architecture hexagonale. Un module par domaine (`candidature`, `tenue`…). Dans chaque module, `domain` contient le modèle, les règles et les **algorithmes** en Java pur, `application` les cas d'utilisation (ports d'entrée), les ports de sortie et les services, et `adapter` les implémentations techniques (REST en entrée, JPA en sortie). Les dépendances pointent vers le domaine, et un test ArchUnit fait échouer le build sinon.

Qualité du code, au-delà de l'hexagone. Clean Code (noms métier identiques dans le code, l'API et ce dossier ; fonctions courtes ; pas de duplication), **SOLID**, objets valeur immuables et invariants vérifiés à la construction, **design patterns** quand ils servent (Adapter et Repository pour les ports, méthodes de fabrique pour les DTO et les données de test), conventions **REST** (201 avec `Location`, 400, 404, 409, `problem+json`), règles **OWASP** (validation, échappement, CORS restreint) et quality gate SonarCloud au vert.

Modèle commun. Pour que les 4 features d'une itération avancent en parallèle, l'équipe fixe dès le départ un petit noyau partagé : le **profil** du candidat, et la **candidature** (entreprise, poste, texte de l'annonce, date, heure et adresse de l'entretien). Chaque feature ajoute ensuite ses propres tables, dans sa propre migration.

06 Démarche pour chaque feature

Les étapes s'enchaînent **dans cet ordre, sans en sauter** : on ne commence une étape que lorsque la précédente est terminée.

ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
1. Idéation	Besoin et scénario d'usage ; **algorithme retenu**, complexité, alternatives écartées ; impact sur le schéma et sur l'API ; critères d'acceptation. On ouvre une issue et on rédige la fiche `doc/features/<version>-F<n>-<nom>.md`.	La fiche est relue et validée par un autre membre.
2. Back	Migration Flyway `V<n>__<feature>.sql` ; **domaine** en Java pur (modèle, règles métier, algorithme) ; **application** (ports d'entrée et de sortie, service) ; adaptateur de persistance.	Le code compile et le test d'architecture (ArchUnit) passe.
3. Tests	Tests **unitaires** de l'algorithme (cas nominaux, cas limites, invariants, propriétés sur données aléatoires) ; services testés avec de faux adaptateurs ; tests d'**intégration** de la persistance sur le vrai schéma.	`./gradlew build` passe, et chaque critère d'acceptation est couvert par au moins un test.
4. API REST	Endpoints sous `/api/v1`, DTO validés, erreurs au format `problem+json`, documentation OpenAPI ; tests d'intégration de l'API.	Les endpoints sont visibles dans Swagger UI et les tests de l'API passent.

ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
5. Front	Écrans qui consomment **uniquement** l'API REST.	Le scénario d'usage se joue de bout en bout ; la pull request est relue ; la fiche est complétée avec son araignée « part IA / part humaine ».

07 Protocole d'usage de l'IA

Principe : l'IA assiste, l'humain reste l'auteur. On ne laisse jamais l'IA tout faire sans comprendre : tout ce qui est commité a été relu et compris par son auteur, capable de l'expliquer, de le modifier et de le refaire sans aide, notamment en soutenance. Les consignes des assistants de code sont versionnées dans `CLAUDE.md`.

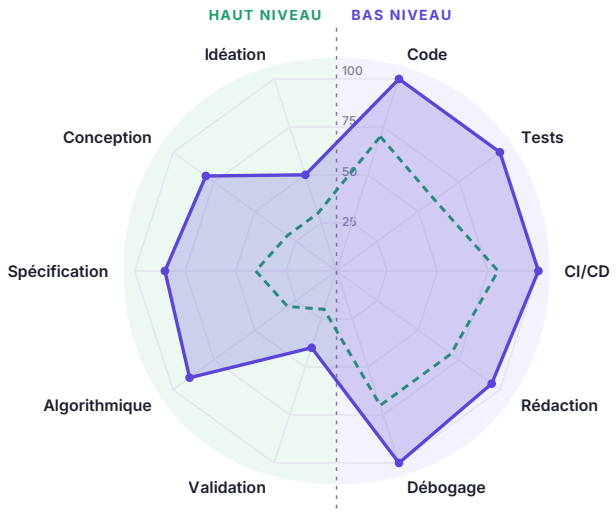
ACTIVITÉ	CE QUE FAIT L'IA	CE QUE FAIT L'HUMAIN
Idéation et conception	Critique les idées, propose des options et des algorithmes candidats	Choisit, justifie le choix et rédige la fiche de la feature
Algorithme (code du domaine)	Explique l'algorithme, relit le code, signale les cas oubliés	**Conçoit et écrit le cœur de l'algorithme** ; sait le dérouler à la main sur un exemple et donner sa complexité
Code technique (adaptateurs, API, configuration, front)	Peut générer un premier jet	Relit chaque ligne, adapte le code au projet, en assume les choix
Tests	Suggère des cas de test	Définit les cas limites et les propriétés ; vérifie qu'un test échoue bien quand on casse volontairement le code
CI/CD, build, conteneurisation	Peut générer workflows, Dockerfile et scripts	Comprend chaque étape, vérifie l'exécution réelle, gère lui-même les secrets
Documentation	Aide à rédiger, structurer et mettre en forme	Valide le contenu et vérifie les faits (chiffres, couverture, résultats)
Débogage et revue	Propose des diagnostics	Reproduit le problème, comprend la cause, corrige

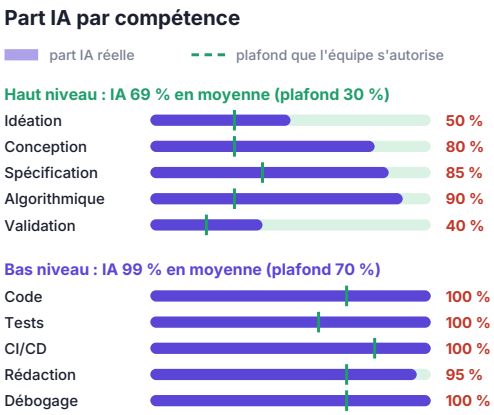
Interdits. Commiter du code qu'on ne comprend pas. Confier à l'IA des secrets (jetons, mots de passe) ou des données personnelles. Laisser l'IA commiter sous une autre identité que celle du membre concerné. Présenter comme humain un travail produit par l'IA.

Traçabilité.

- Chaque pull request comporte une section « Usage de l'IA » : outil utilisé, objet de la demande, ce qui a été repris, modifié ou rejeté. Elle est relue par un autre membre.
- La part de l'IA est déclarée **par compétence** dans le graphe en araignée **demandé par le cours** (annexe), car une mesure unique serait biaisée : faire écrire du code par l'IA n'a pas le même poids que lui faire concevoir. Les compétences de **haut niveau** (idéation, conception, spécification, algorithmique, validation) sont séparées de celles de **bas niveau** (code, tests, CI/CD, rédaction, débogage), et chaque axe compare la part IA réelle à un **plafond** que l'équipe s'autorise, plus bas pour le haut niveau. Les valeurs sont mesurées en lignes d'après les PR quand c'est possible, estimées sinon. Une araignée accompagnera chaque feature en 0.1.0, 0.2.0 et 1.0.0.
- Exemple de critique d'idée obtenue avec l'IA : `exemples-ia/`.

08 Annexe : part IA / part humaine





État au 20 octobre 2026, pour le dossier de lancement et les deux premières features de la v0.1.0. Les valeurs sont dans `doc/part-ia.json`, et le graphe est régénéré à chaque compilation du dossier. **Constat :** sur ce démarrage, la part IA dépasse le plafond visé sur tous les axes, et surtout sur l'algorithmique et la conception. Les prochaines features devront rééquilibrer en priorité les compétences de haut niveau.